

Detalle armario

COMPONENTES Y CONEXIONES

Contenidos

Comentarios y recomendaciones	2
Conexión y subida de código	2
Comunicación Serie	2
Reset y monitor serie	2
Caudalímetros	3
Bombas secundarias	3
Sensores de temperatura del agua (Wtemp)	3
Referencias de conexiones en tablero	4
Pines utilizados	4
Conexiones actuales y auxiliares	4
Arduino	0
Comentarios	1
Comunicación Serie	1
Reset y monitor serie	1
Caudalímetros	1
Bombas secundarias	2
Sensores de temperatura del agua (Wtemp)	2
Explicación del código	2
Bloque de cabecera	3
Parámetros de configuración	3
Librerías, pines y variables internas	3
Configuración de sensores, variables de estado y PID	4
setup()	4
loop()	4
Función vent_config()	5
Función bombas_config()	6
Función LED_config()	6
Función caudalimetro_config()	6
Función temp_config()	7
Función b1_on(float ml1)	7

Función LED(unsigned long LED_tOn, unsigned long LED_tOff)	7
Función temp_control(float setPoint, unsigned long temp_time)	8
Función Wtemp_read(unsigned long Wtemp_time)	9
Función bomba(unsigned long bomba_tOn, unsigned long bomba_tOff)	9
Función vent(bool vent_st)	9
Función de lectura de pH (pH_read())	9
Función de lectura de EC (EC_read())	10
Funciones de control de pH y EC (pHcontrol() y ECcontrol())	10

Comentarios y recomendaciones

Conexión y subida de código

Al momento de conectar la placa Arduino a la computadora mediante USB, se recomienda que el tablero se encuentre energizado desde su fuente de alimentación. De este modo se evita que la corriente proveniente del puerto USB intente alimentar indirectamente a los demás componentes del tablero, lo que podría generar comportamientos inestables o incluso daños en la computadora o el armario.

Comunicación Serie

Se acordó con los estudiantes de sistemas que el puerto serie solo debe usarse para enviar el JSON. Esto evita interferencias o reinicios inesperados al comunicarse con la Raspberry Pi.

En caso de necesitar imprimir información adicional para pruebas u otros fines, la comunicación con la Raspberry Pi debe pausarse o interrumpirse temporalmente.

Reset y monitor serie

Inicialmente se agregó un capacitor de 10 μ F entre el pin GND y RESET para evitar que el Arduino se reiniciara al abrir el monitor serie. Posteriormente se decidió retirarlo, ya que impedía subir nuevos códigos correctamente.

Se evaluó la situación junto con los estudiantes de sistemas y se concluyó que no sería un problema abrir y cerrar el monitor serie con la configuración actual, por lo que el capacitor no es necesario.

Para referencia futura, si fuese necesario evitar reinicios, podría implementarse un interruptor que active o desactive un capacitor, permitiendo controlar cuándo se reinicia el Arduino sin afectar la carga de programas.

Caudalímetros

Se comprobó el funcionamiento del caudalímetro 1 y se determinó la ecuación que relaciona los pulsos con el volumen de riego de su bomba:

```
vol1 = ((pulsos1 * VOLUMEN_POR_PULSO) / 10.0)-10.7;
```

Los caudalímetros 2 y 3 utilizan la misma ecuación, aunque podrían incorporarse factores de corrección específicos:

- Caudalímetro 2: offset 10.8
- Caudalímetro 3: offset 10.9

Actualmente, todas las funciones de riego utilizan un offset de 10.7, pero cada caudalímetro puede tener su propio término de corrección dentro de su función, sumando o restando según corresponda.

Bombas secundarias

Todas las bombitas funcionan correctamente. Las bombas 3 y 4 fueron reemplazadas en septiembre de 2025.

La cantidad de riego que debe suministrar cada bomba se calcula en función de las mediciones de pH y conductividad eléctrica (EC). Cuando los valores de pH o EC se desvían de sus rangos aceptables, las funciones de control (pHcontrol y ECcontrol) determinan cuánto volumen debe aportar cada bomba para corregir los niveles, utilizando la ecuación de cada caudalímetro y sus offsets correspondientes.

Sensores de temperatura del agua (Wtemp)

Se colocaron dos sensores: uno en el estante superior y otro en el inferior. De esta forma se monitorea la temperatura de la solución en ambos niveles.

Opcionalmente, podría colocarse un sensor directamente en el tanque principal, aunque también es posible estimar la temperatura del tanque observando la del estante superior unos minutos después de iniciar la recirculación del agua.

Referencias de conexiones en tablero

Pines utilizados

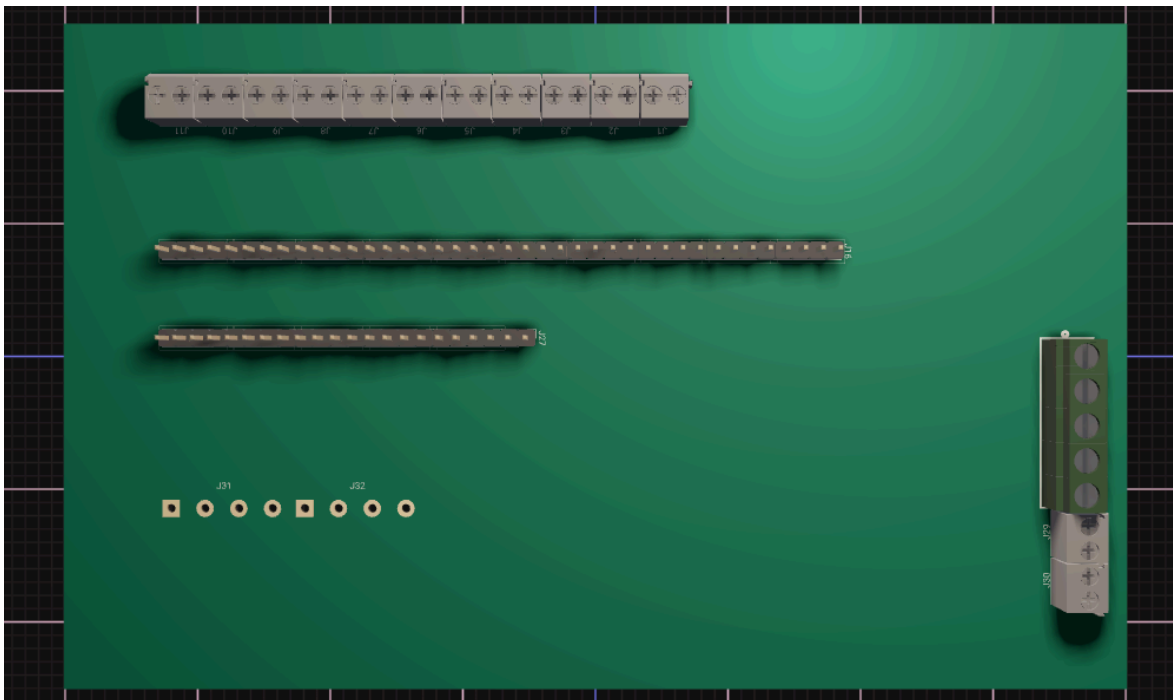
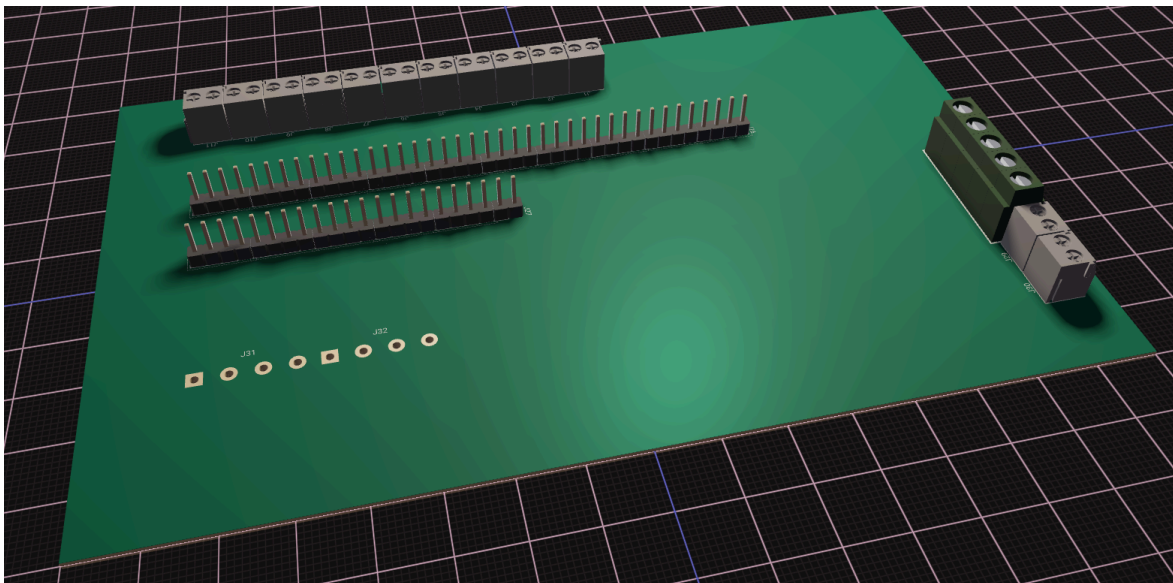
Se presenta a continuación, extraído del código de referencia, una lista de pines utilizados.

```
// =====
//          MAPA DE PINES - HARDWARE
// =====
// | Dispositivo          | Nombre      | Pin | Descripción      |
// -----
// | Ventilador LED       | v0          | 29  | Ventilador LED   |
// | Ventilador 1         | v1          | 23  | Ventilador lateral |
// | Ventilador 2         | v2          | 25  | Ventilador lateral |
// | Ventilador 3         | v3          | 27  | Ventilador lateral |
// -----
// | Bomba principal      | b0          | 43  | Tanque principal |
// | Bombita tanque 1     | b1          | 31  | Riego tanque 1   |
// | Bombita tanque 2     | b2          | 33  | Riego tanque 2   |
// | Bombita tanque 3     | b3          | 35  | Riego tanque 3   |
// | Bombita tanque 4     | b4          | 37  | Riego tanque 4   |
// -----
// | LED estante arriba   | LED1        | 39  | Iluminación      |
// | LED estante abajo    | LED2        | 41  | Iluminación      |
// -----
// | Caudalímetro 1       | c1          | 45  | Tanque 1         |
// | Caudalímetro 2       | c2          | 47  | Tanque 2         |
// | Caudalímetro 3       | c3          | 49  | Tanque 3         |
// | Caudalímetro 4       | c4          | 51  | Tanque 4         |
// -----
// | DHT22 exterior       | DHTPIN_ext  | 2   | Temp/Hum exterior |
// | DHT22 interior       | DHTPIN_int  | 3   | Temp/Hum interior |
// | DS18B20              | Wtemp_pin   | 4   | Temp solución    |
// -----
// | LDR                  | LDRpin      | A0  | Luz ambiente     |
// | Sensor pH            | pHPin       | A1  | Medición pH      |
// | Sensor EC            | ECPin       | A2  | Medición EC      |
// =====
```

Conexiones actuales y auxiliares

El tablero cuenta con una placa perforada ubicada en la parte central, a la derecha del Arduino, donde se soldaron diversos bornes y pines. Muchos de estos puntos no se utilizaron y quedaron disponibles para futuras conexiones o ampliaciones del sistema.

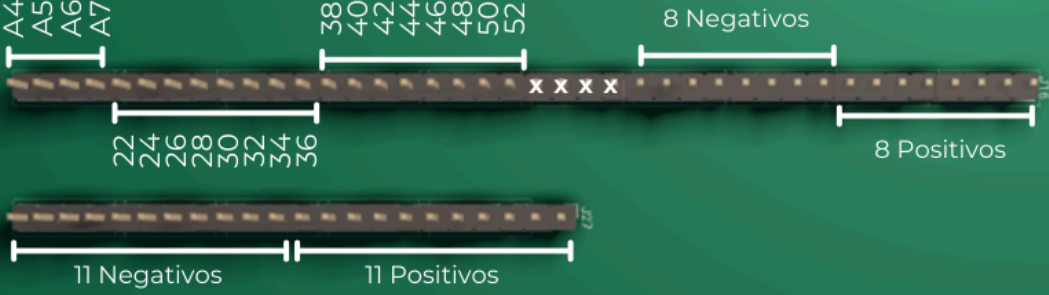
A continuación, se presenta una imitación de la ubicación de los distintos componentes, con dos vistas distintas, y en la ultima con las respectivas referencias.



5V

- Reset
- 3,3 V
- 5V
- GND
- Vin 12V

J31 J32



- LDR A0
- pH A1
- EC A2
- A3
- 0
- 1
- DHT22 2
- DHT22 3
- DS18B20 4
- 5
- Cauda 1 45
- Cauda 2 47
- Cauda 3 49
- Cauda 4 51
- 53
- 54
- 55
- 56
- 57
- 58
- 59
- 60
- 61
- 62
- 63
- 64
- 65
- 66
- 67
- 68
- 69
- 70
- 71
- 72
- 73
- 74
- 75
- 76
- 77
- 78
- 79
- 80
- 81
- 82
- 83
- 84
- 85
- 86
- 87
- 88
- 89
- 90
- 91
- 92
- 93
- 94
- 95
- 96
- 97
- 98
- 99
- 100

4 Negativos 5 Positivos

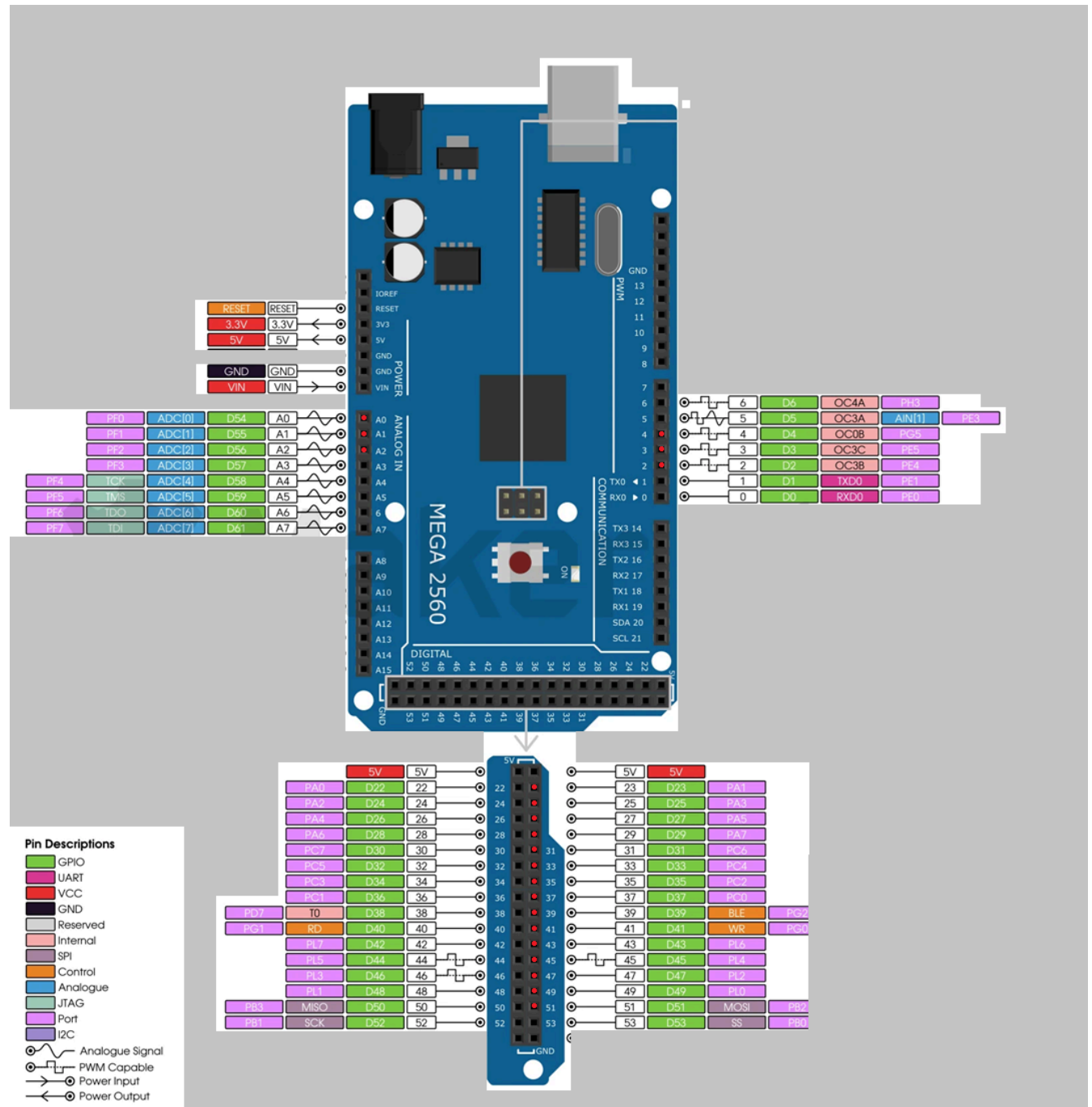


12V

Arduino

Pines con conexiones ya realizadas o disponibles en la placa perforada:

- En rojo: pines actualmente utilizados.
- Los restantes que se muestran con descripción están cableados y disponibles en la placa del tablero.



Explicación del código

El proyecto está organizado en dos grandes secciones:

Parámetros de configuración

- En esta parte se encuentran todas las variables que el usuario puede y debe modificar para ajustar el funcionamiento del sistema a su instalación concreta.
- Aquí se incluyen tiempos de muestreo, umbrales, consignas de temperatura, rangos aceptables de pH, límites de conductividad eléctrica (EC), etc.
- Es la única parte pensada para ser modificada de manera regular, por lo que cada parámetro está acompañado de un comentario explicativo que indica su función.

Lógica de funcionamiento

- Esta parte contiene todo el código que hace que el sistema funcione automáticamente: lectura de sensores, aplicación de controles (on/off o PID), activación de bombas, ventiladores, electroválvulas y demás actuadores.
- No se recomienda modificar esta sección, salvo que se desee cambiar la lógica de control del sistema (por ejemplo, pasar de un control on/off a uno proporcional, o cambiar la forma en que se dosifica ácido o base para el pH).

De esta manera, el usuario que simplemente quiera usar el sistema solo necesita revisar la primera sección (parámetros). Quien quiera experimentar o extender funcionalidades puede adentrarse en la lógica de funcionamiento.

Bloque de cabecera

En esta sección, completa en forma de comentario, se detallan algunas de las características ya mencionadas del código y posteriormente se muestra una tabla que representa el cableado de los componentes. En la misma se indica, de los pines en uso, a que corresponde cada uno.

Parámetros de configuración

En esta sección se incluyen todos los valores que el usuario puede modificar para personalizar el funcionamiento del sistema. Esto abarca límites de humedad y temperatura, rangos de pH y conductividad eléctrica, así como tiempos de operación de ventiladores, LEDs y bombas. Modificar estos parámetros permite ajustar el comportamiento del sistema según las condiciones específicas del cultivo o del entorno.

Respecto a los tiempos de operación y medición:

- Sensores: Para cada sensor (pH, EC, temperatura ambiente, temperatura de la solución y luz ambiente) se indica cada cuánto tiempo se realiza la medición.
- Actuadores: Para los dispositivos como LEDs y la bomba principal se define cuánto tiempo permanecen encendidos y apagados.

Todo el control del sistema se organiza en base a estos rangos de tiempo.

Librerías, pines y variables internas

A partir de esta sección se definen todos los elementos internos necesarios para el funcionamiento del sistema. Esto incluye:

- Librerías: Se incluyen las librerías necesarias para manejar los sensores (DHT22 para temperatura y humedad, DS18B20 para temperatura de la solución) y para el manejo de datos en formato JSON.
- Declaraciones de pines: Se asigna un pin específico de la placa a cada componente del sistema:
- Organización de entradas y salidas: Se diferencian claramente las salidas digitales (actuadores) de las entradas digitales (sensores).

Esta sección no debe ser modificada por el usuario, salvo que se quiera cambiar la lógica interna del sistema o reasignar algún pin.

Configuración de sensores, variables de estado y PID

En la configuración de sensores se definen e inicializan los distintos dispositivos de medición que forman parte del sistema. Se incluyen los sensores DHT22, encargados de obtener la temperatura y humedad tanto en el interior como en el exterior; el sensor LDR, que mide la iluminación ambiente; los DS18B20, destinados a registrar la temperatura del agua; y los sensores de pH y EC, que cuentan además con parámetros de calibración y compensación para asegurar lecturas correctas en distintas condiciones. También se establece el tamaño del buffer para el manejo de datos en formato JSON, lo que permitirá estructurar la información antes de transmitirla o procesarla.

A continuación, en la sección de variables de estado, se declaran las variables que representan el estado actual de los distintos actuadores y caudalímetros. Para las bombas, ventiladores y LEDs se utilizan valores booleanos que indican si cada uno está encendido o apagado, mientras que para los caudalímetros se guardan tanto los pulsos detectados como los volúmenes de agua medidos y los volúmenes objetivo de riego. De esta manera, el sistema puede llevar un seguimiento del funcionamiento de cada componente en tiempo real.

Por último, se definen los parámetros del controlador PID, que incluyen las ganancias proporcional, integral y derivativa, junto con las variables de error, integral acumulada, salida y registros del ciclo anterior.

setup()

En esta sección se inicializan todos los elementos del sistema antes de comenzar el bucle principal. Primero se abre la comunicación serie y luego se configuran los periféricos mediante funciones específicas definidas más adelante en el código. Estas funciones asignan los pines como entradas o salidas y establecen sus estados iniciales (por ejemplo, ventiladores, bombas y LEDs en LOW).

También se configura la lectura de los sensores: caudalímetros, sensores de temperatura y potenciómetros (simulando los sensores de pH y conductividad eléctrica). Finalmente, se guardan variables iniciales necesarias para el control PID, como el tiempo de referencia (lastTime) y la primera medida de temperatura.

loop()

Dentro del bucle principal se ejecuta la lógica repetitiva del sistema. Aquí se llaman distintas funciones que controlan sensores y actuadores, cada una con parámetros configurables definidos previamente:

- LED(): controla los ciclos de encendido y apagado de la iluminación.
- temp_control(): gestiona la lectura de temperatura y la activación de ventiladores (con control PID).
- Wtemp_read(): mide periódicamente la temperatura del agua.
- bomba(): controla los tiempos de encendido y apagado de la bomba principal.

También están incluidas, aunque comentadas, funciones para el control de pH, conductividad eléctrica y bombas de dosificación de cada tanque.

Además, el loop() contiene un bloque para imprimir datos:

Se usa un temporizador (millis()) para mostrar en el puerto serie variables de interés a intervalos regulares (solo para depuración).

Si se recibe el comando 'P' por el puerto serie, se construye un objeto JSON que organiza la información del sistema en diferentes categorías:

- Mediciones de sensores: temperatura y humedad (interior y exterior), temperatura de solución, pH, conductividad eléctrica y nivel de luz.
- Setpoints: valores de referencia configurados (ej. temperatura límite, rangos de pH y EC).

- Intervalos de tiempo: cada cuánto se mide o se activa un dispositivo (bombas, LEDs, sensores).
- Estados: indica si cada actuador está encendido o apagado (bombas, ventiladores, LEDs).
- Fallos: por ejemplo, si el control PID presenta un error.

Una vez armado, este JSON se convierte en un string y se envía por el puerto serie. De esta forma, la Raspberry Pi u otro dispositivo externo recibe un paquete de datos estructurado y legible, listo para almacenar, visualizar o procesar.

Función `vent_config()`

Esta función inicializa los pines asociados a los relés que controlan los ventiladores:

- Primero configura cada pin (v0, v1, v2, v3) como salida con `pinMode(...)`.
- Luego asegura que todos los ventiladores arranquen apagados poniendo esos pines en LOW mediante `digitalWrite(...)`.
- Finalmente, actualiza las variables de estado (v0_state, v1_state, etc.) y las inicializa en 0, indicando que cada ventilador está en estado apagado al inicio.

De esta forma, el sistema siempre arranca en una condición segura y conocida (ningún ventilador encendido).

Función `bombas_config()`

Esta función configura los pines de salida que controlan las bombas:

- Define como salida cada pin correspondiente a las bombitas dosificadoras (b1, b2, b3, b4) y a la bomba principal (b0).
- Inicializa todas las bombas apagadas: las dosificadoras en LOW y la bomba principal en HIGH, porque en este caso el relé está cableado de manera que HIGH corresponde a apagado.
- Actualiza las variables de estado (b1_state, b2_state, etc.) para reflejar que todas las bombas están inicialmente apagadas.

Así se asegura que el sistema arranque sin dosificación ni circulación de agua accidental.

Función `LED_config()`

Esta función configura los pines de salida que controlan las luces LED:

- Define como salida los pines LED1 y LED2, correspondientes a los estantes de cultivo.

- Inicializa ambos pines en HIGH, que en este caso significa apagado (debido a la lógica inversa del relé sólido).
- Ajusta la variable de estado LED_state en 0, indicando que ambos estantes empiezan apagados.

Esto garantiza que las luces no se enciendan automáticamente al iniciar el sistema, manteniendo un arranque controlado.

Función caudalimetro_config()

Se encarga de configurar los pines conectados a los caudalímetros como entradas digitales:

- c1: Caudalímetro de la bomba de pH A.
- c2: Caudalímetro de la bomba de pH B.
- c3: Caudalímetro de la bomba de nutrientes 1.
- c4: Caudalímetro de la bomba de nutrientes 2.

Esto permite que el microcontrolador reciba las señales de pulsos generadas por los caudalímetros para medir el caudal de cada dosificación.

Función temp_config()

Inicializa los sensores de temperatura:

- Configura como entradas los pines de los sensores DHT22 (DHTPIN_int y DHTPIN_ext) y los activa con begin(), lo que habilita la lectura de temperatura y humedad interna y externa.
- Configura también como entrada el pin del sensor DS18B20 (Wtemp_pin) y lo inicializa con DS18B20.begin(), lo que habilita la lectura de la temperatura del agua.

De esta forma, el sistema queda listo para registrar temperatura y humedad tanto en el ambiente como en la solución nutritiva.

Función b1_on(float ml1)

Esta función b1_on(float ml1) controla la dosificación de la bombita 1 usando como parámetro el volumen deseado en mililitros (ml1).

La lógica es la siguiente:

- Mientras el volumen medido (vol1) sea menor al deseado, se cuentan los pulsos que envía el caudalímetro 1 mediante un interrupción, y se calcula el volumen que ya pasó.

- Durante este proceso, la bomba 1 se mantiene encendida.
- Cuando se alcanza el volumen objetivo, se apaga la bomba, se detiene la medición de pulsos y se reinician las variables de volumen y pulsos.

El mismo esquema se aplica a las demás bombas (b2_on, b3_on, b4_on), cada una usando su propio caudalímetro y volumen objetivo. Esto permite controlar la cantidad de líquido que se suministra a cada tanque.

Función LED(unsigned long LED_tOn, unsigned long LED_tOff)

La función LED(unsigned long LED_tOn, unsigned long LED_tOff) controla el encendido y apagado de los LEDs de los estantes en ciclos de tiempo definidos por los parámetros LED_tOn (tiempo encendidos) y LED_tOff (tiempo apagados).

La lógica es la siguiente:

- Se verifica cuánto tiempo ha pasado desde el último cambio de estado (previousMillis_LED).
- Si los LEDs estaban apagados y ha pasado el tiempo de apagado (LED_tOff), se encienden los LEDs y el ventilador LED (v0), y se reinicia el contador de tiempo.
- Si los LEDs estaban encendidos y ha pasado el tiempo de encendido (LED_tOn), se apagan los LEDs y el ventilador LED, y se reinicia nuevamente el contador de tiempo.
- Además, se lee el sensor LDR para estimar el nivel de iluminación y, opcionalmente, detectar fallos si los valores no coinciden con el estado esperado de los LEDs.

Este enfoque permite un control cíclico de la iluminación sin bloquear la ejecución del resto del sistema, usando el tiempo transcurrido en lugar de delay().

Función temp_control(float setPoint, unsigned long temp_time)

La función temp_control(float setPoint, unsigned long temp_time) se encarga de medir la temperatura y la humedad y de controlar los ventiladores de manera On/Off, siguiendo estos pasos:

1. Temporización:
 - a. Solo se ejecuta si ha pasado el tiempo definido por temp_time desde la última medición, usando millis(). Esto evita lecturas constantes y permite trabajar por intervalos regulares.
2. Lectura de sensores:
 - a. Se leen las temperaturas y humedades del interior (dht_int) y del exterior (dht_ext).

- b. Los valores medidos se almacenan en variables globales (temp_value_int, hum_value_int, etc.).
- 3. Lógica On/Off de control de ventiladores:
 - a. Se compara la temperatura interior con la consigna (setPoint), pero solo si la temperatura exterior es menor que la consigna, ya que no tendría sentido ventilar si la temperatura exterior ya es mayor.
 - b. Se usa un contador de muestras (temp_samples) para evitar que el ventilador prenda y apague continuamente ante pequeñas variaciones de temperatura (histéresis de ± 1 °C).
 - c. Cuando el contador alcanza un valor absoluto de 3, se activa o desactiva el ventilador según corresponda y se resetea el contador.
- 4. Bloque PID (comentado):
 - a. Originalmente, la función implementaba un control PID, calculando un output proporcional a la diferencia entre la temperatura objetivo y la medida.
 - b. En esta versión se mantiene comentado para simplificar y usar solo el control On/Off.

Función Wtemp_read(unsigned long Wtemp_time)

La función Wtemp_read(unsigned long Wtemp_time) se encarga de medir la temperatura del agua en los dos estantes del sistema hidropónico. La lectura solo se realiza si ha pasado el intervalo definido por Wtemp_time desde la última medición, usando millis(), lo que permite muestreos periódicos sin bloquear el resto del programa.

Se solicita a los sensores DS18B20 que actualicen sus mediciones con requestTemperatures(), y luego se leen las temperaturas de cada estante usando getTempCByIndex(0) y getTempCByIndex(1), almacenándolas en Wtemp0_value (estante inferior) y Wtemp1_value (estante superior).

Función bomba(unsigned long bomba_tOn, unsigned long bomba_tOff)

La función bomba(unsigned long bomba_tOn, unsigned long bomba_tOff) controla la recirculación del fluido en el sistema hidropónico. Funciona alternando el encendido y apagado de la bomba principal según los intervalos de tiempo definidos por bomba_tOn y bomba_tOff.

Usando millis(), la función verifica si ha transcurrido el tiempo necesario desde el último cambio de estado. Si la bomba estaba encendida (b0_state == HIGH) y se cumplió el

tiempo de encendido, se apaga; si estaba apagada (`b0_state == LOW`) y se cumplió el tiempo de apagado, se enciende.

Esto permite que la bomba recircule el fluido de manera periódica sin bloquear la ejecución del resto del programa.

Función `vent(bool vent_st)`

La función `vent(bool vent_st)` controla los ventiladores del sistema para regular la temperatura.

Recibe un parámetro `vent_st` que indica si los ventiladores deben encenderse (1) o apagarse (0). Todos los ventiladores se controlan de manera conjunta, aunque podrían separarse si se quisiera un control independiente. La función simplemente cambia el estado físico de los pines de los ventiladores y actualiza las variables de estado correspondientes (`v1_state`, `v2_state`, `v3_state`). Esto permite que la lógica de control de temperatura pueda activar o desactivar la ventilación de forma sencilla.

Función de lectura de pH (`pH_read()`)

Esta función permite medir el pH de la solución hidropónica a partir del sensor conectado al pin `pHPin`. Primero se obtiene el valor analógico crudo mediante `analogRead(pHPin)`, que se convierte a voltaje usando la referencia del Arduino: $\text{voltage} = \text{raw} * (5.0 / 1023.0)$. Luego se aplica una fórmula de aproximación $\text{pH_value} = 3.5 * \text{voltage} + \text{offset}$ para calcular el valor de pH. El parámetro `offset` se utiliza para ajustar la medición según la calibración real del sensor, y el resultado final se devuelve al llamarla.

Nota de uso: Para obtener mediciones precisas, el sensor debe calibrarse con soluciones estándar de pH conocido. La calibración permite ajustar el parámetro `offset` para que la lectura refleje correctamente el valor real de pH. Sin calibración, los valores obtenidos pueden ser incorrectos.

Función de lectura de EC (`EC_read()`)

Esta función permite medir la conductividad eléctrica (EC) de la solución hidropónica mediante el sensor conectado al pin `ECPin`. Primero se obtiene la lectura analógica cruda con `analogRead(ECPin)` y se convierte a voltaje aplicando la referencia del Arduino: $\text{voltage} = \text{raw} * \text{VREF} / 1024.0$.

Luego se ajusta este voltaje usando un factor de calibración (`coef`) para obtener `ECraw`. Como la EC depende de la temperatura, se aplica un coeficiente de compensación por temperatura: $\text{TempCoefficient} = 1.0 + \text{tempCoef} * (\text{temp_value_int} - \text{refTemp})$.

Finalmente, la EC corregida se calcula como $EC_value = EC_{raw} / TempCoefficient$ y se devuelve el valor.

Nota de uso: Al igual que el sensor de pH, el sensor de EC requiere calibración con soluciones patrón para asegurar que el valor devuelto refleje correctamente la conductividad real. Los parámetros coef, VREF y tempCoef se deben ajustar según la calibración y la temperatura de referencia.

Funciones de control de pH y EC (pHcontrol() y ECcontrol())

Estas funciones permiten mantener los niveles de pH y conductividad eléctrica (EC) dentro de los rangos aceptables definidos en los parámetros de configuración (pH_low, pH_high, EC_low, EC_high).

El funcionamiento es el siguiente:

1. Cada función mide su respectivo valor mediante las funciones pH_read() o EC_read().
2. Se comparan las lecturas con los límites configurados. Si el valor está fuera del rango, se aumenta o disminuye un contador de muestras (pH_samples o EC_samples). Esto sirve para evitar reacciones por lecturas aisladas y asegurar que el cambio sea consistente.
3. Cuando se acumulan 3 muestras consecutivas que indican que se necesita corrección, se activa la bomba correspondiente para ajustar el pH o la EC:
 - a. Para pH: b1_on() si está alto, b2_on() si está bajo.
 - b. Para EC: b3_on() si está alto, b4_on() si está bajo.
4. Tras la acción correctiva, se reinicia el contador de muestras.

Nota: La misma lógica se aplica tanto para el control de pH como para EC; lo único que cambia son los sensores y las bombas que se activan para la corrección.